How machines “Learn” with matrices

- From Neurons to Networks
- Perceptions as logistic regression
- CNNs, RNNs, and Transformer architecture

The Jackson Laboratory

This is a slide saver: With Seamus' advice to focus on a "hook" for the course, in module 2, that is probably building on the sentence in module 1 with the addition:

(Mechanism/nuts & bolts of alignment problem): The loss function is where alignment enters – if you measure the wrong thing, the model optimizes towards the wrong goal!*

* I will mention this in module 1, but it is the type of thing that might benefit from being echoed at the beginning (?) of each module to try to emphasize the

Learning Goals:

What are the different types of AI model and how are they used

You will be able to:

- Break down a model into its basic layers
- Identify what type of layer sets each model apart
- Understand the purpose of each type of layer

Neural Networks: Perceptrons & Fully connected Networks

Not enough regression

Learning Goals:

What are neural networks and how do they work?

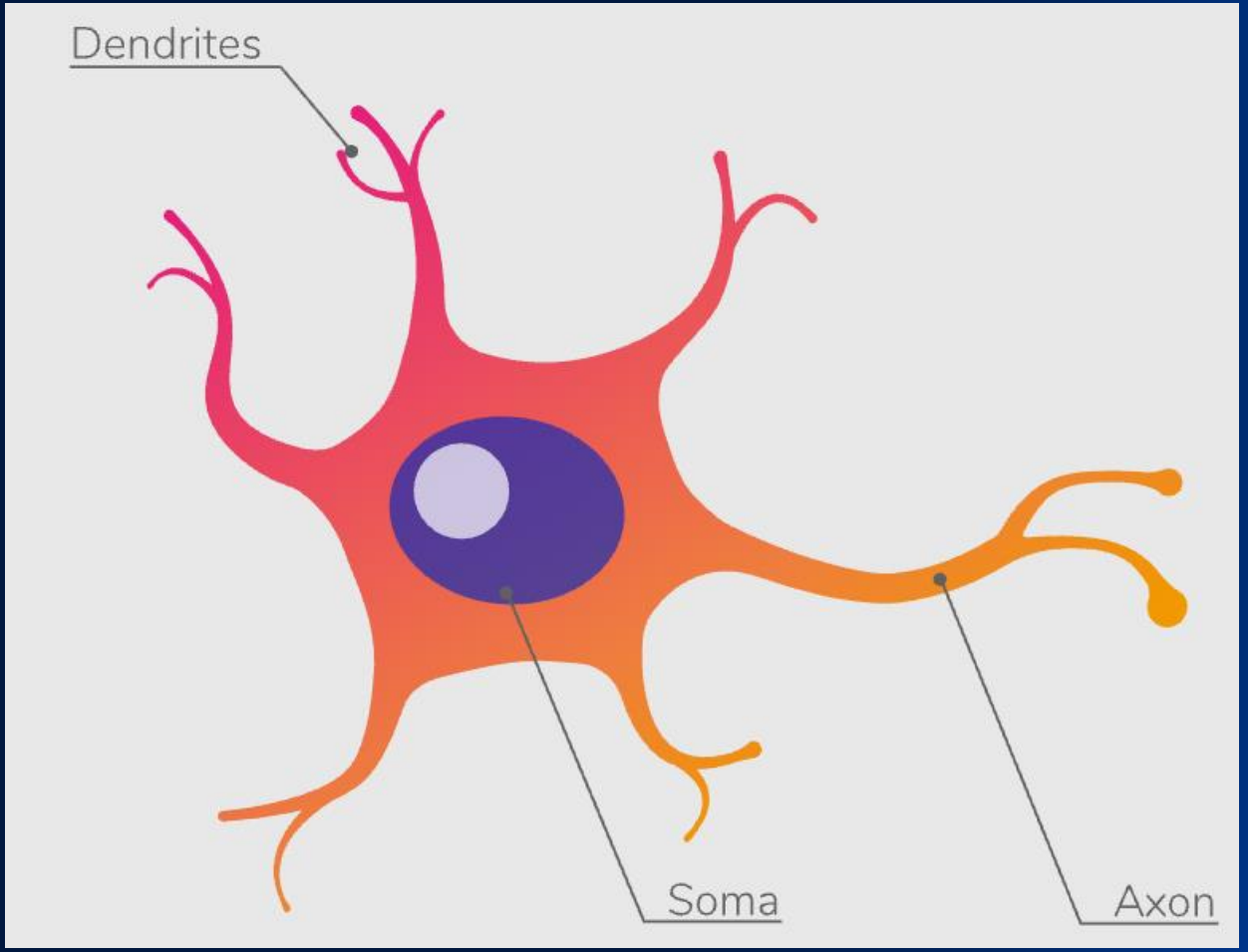
You will be able to:

- Explain what a perceptron is
- Combine perceptrons into a fully connected layer

A brief aside: Biological Neurons

Neurons are the foundational unit of the nervous system.

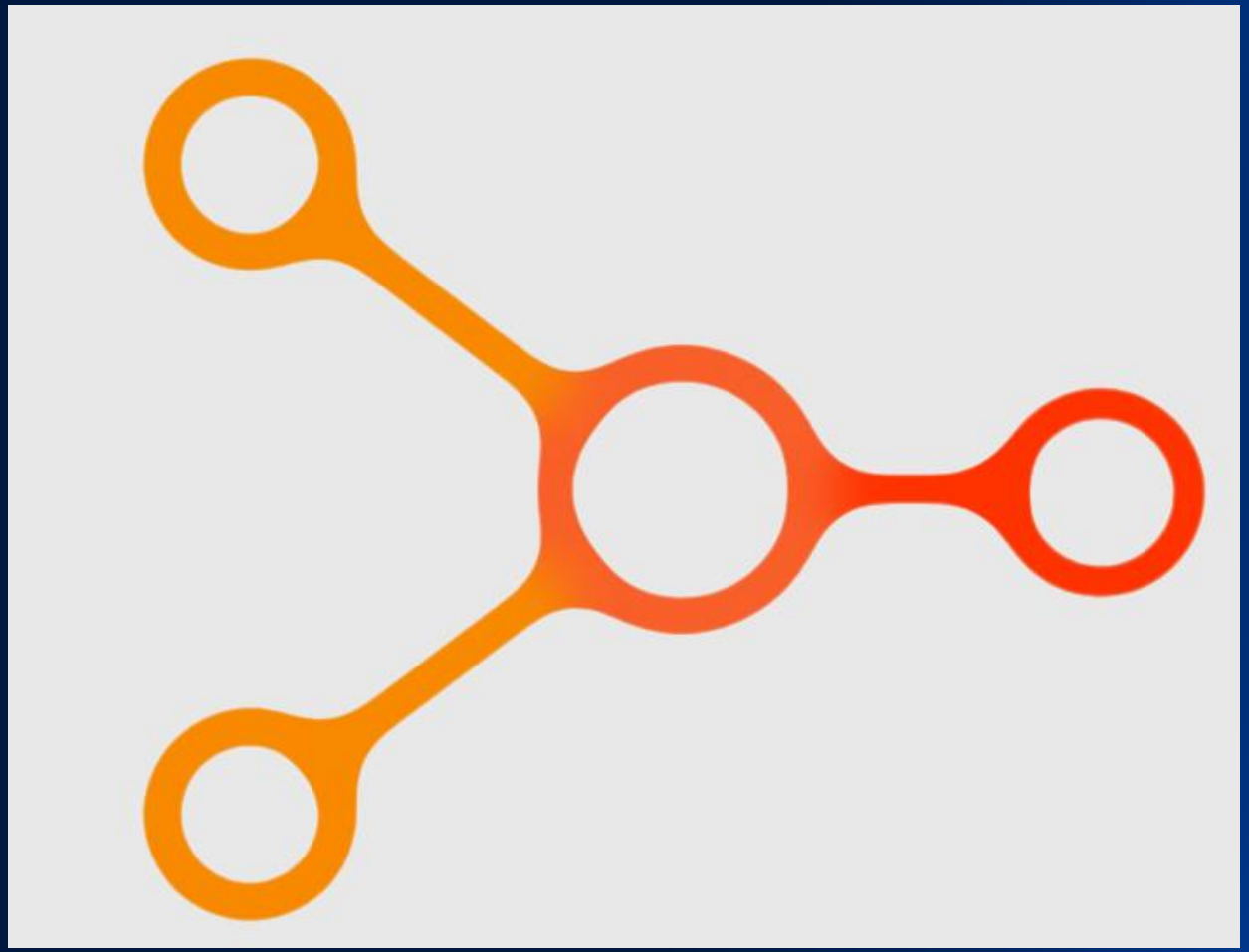
- Dendrites take in signals from other cells
- The cell body/soma processes these signals and decides whether to “fire”
- When the neuron fires a signal is sent through the axon to other cells



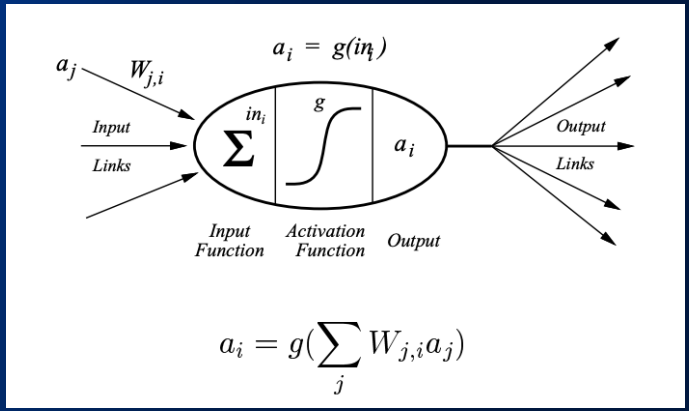
Can we create a mathematical model based on neurons?

Perceptrons are the foundational unit of neural networks.

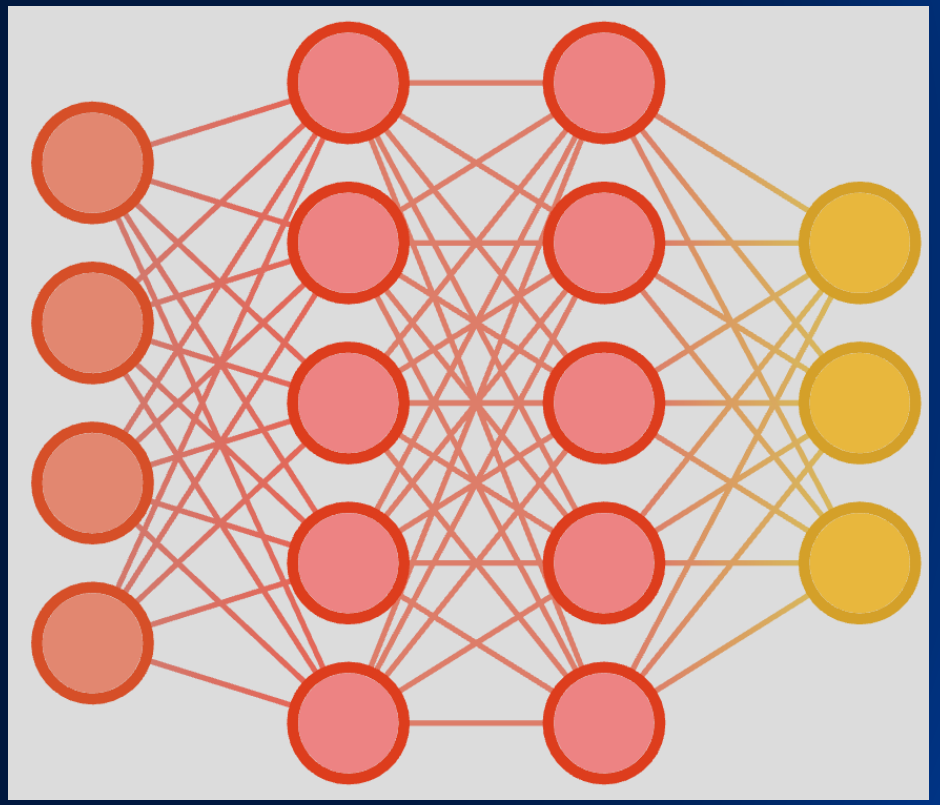
- A perceptron takes one, or (usually) more, inputs.
- These inputs are combined, using a weighted sum, into a single output.
    - Important term: “Linier Algebra”
- This output can then be modified by an activation function
- You may have noticed the behavior of an individual perceptron is basically a regression model like we saw yesterday



We've recreated regression, so what?



Age	Glucose	Has Diabetes (Yes = 1, No = 0)
40	130	1
25	100	0



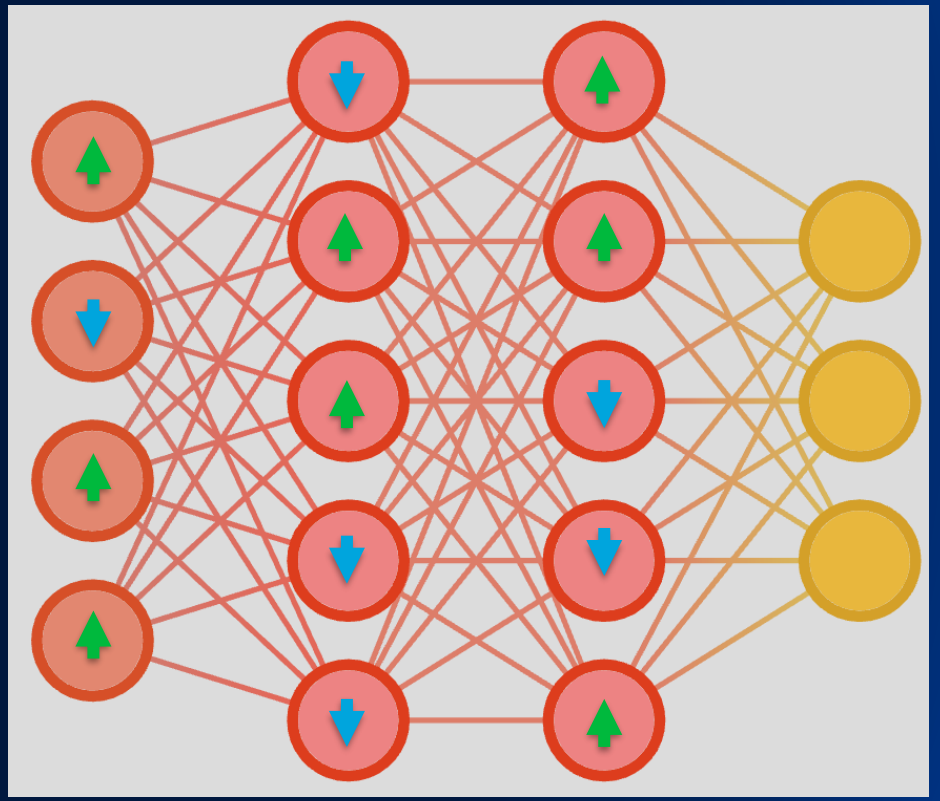
The brain is not made up of just one neuron

- By connecting multiple perceptrons together we create an artificial neural network
  - Two important terms: Fully connected & Feed forward
- Artificial neural networks can solve much more complex problems than "simple" regression

How do we learn the weights?

Remember gradient decent?

- We now have a much more complex function
- The chain rule from calculus let's us break up a complex derivative
- We can see how each perceptron is contributing to the gradient and determine how to address its weights
- This is called "Back propagation"



To discuss after the activity

This activity demonstrates how a fully connected network operates

There was one part of this exorcise will look ahead to our next topic. Can anyone identify it?

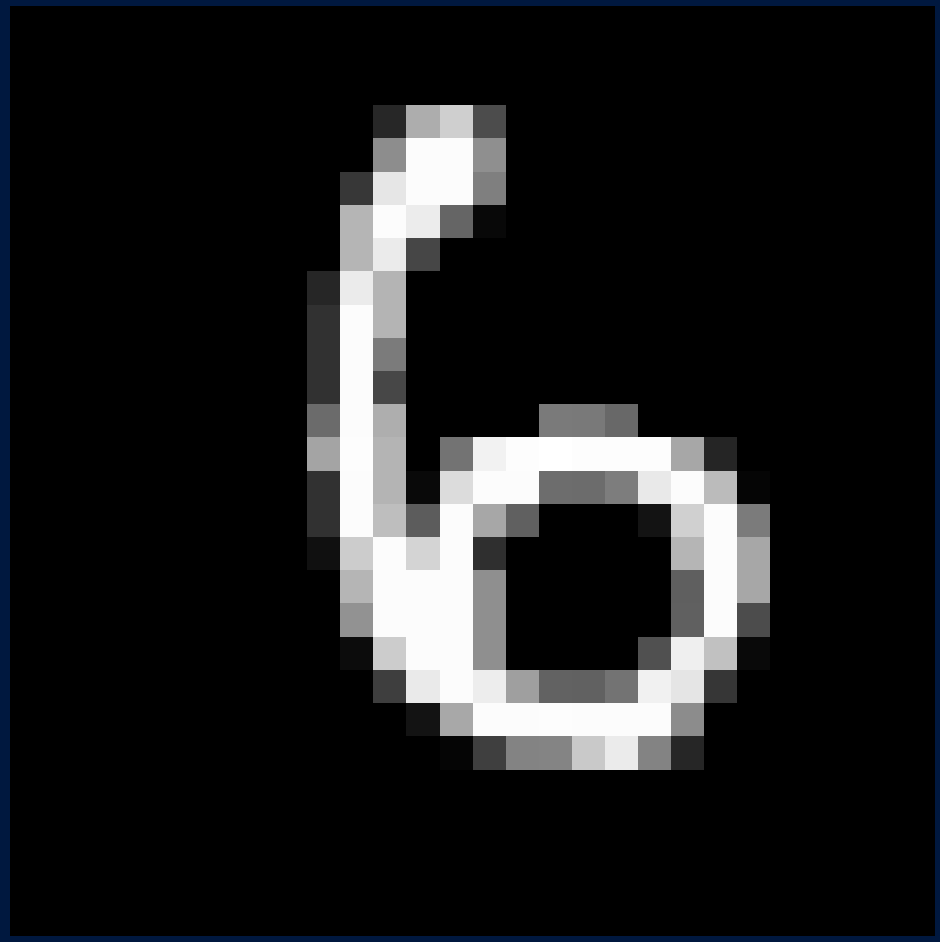
Can you think of any ways to improve our HNN’s performance?

Human Neural Network Activity

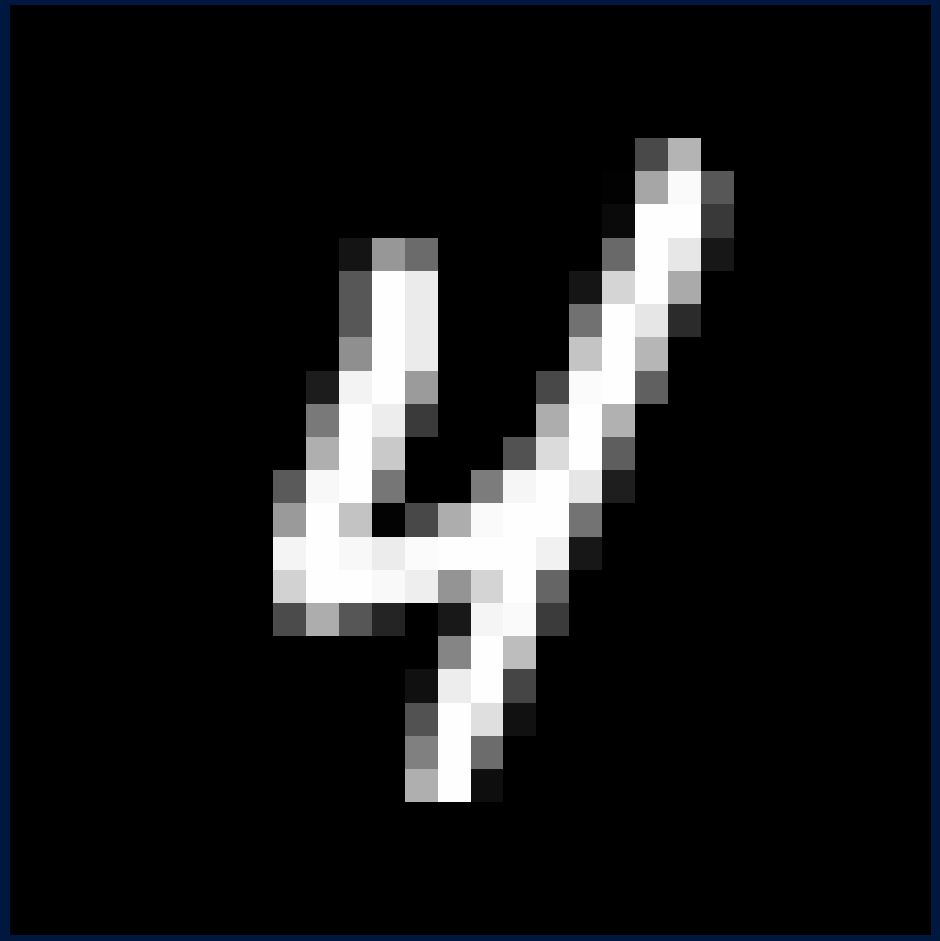
1. We need five nodes for our input layer, five nodes for the hidden layer, and one node to serve as our output layer.
    - One person can serve as multiple nodes or multiple people can team up for a single node depending on attendance

2. See Hand out for specific instructions

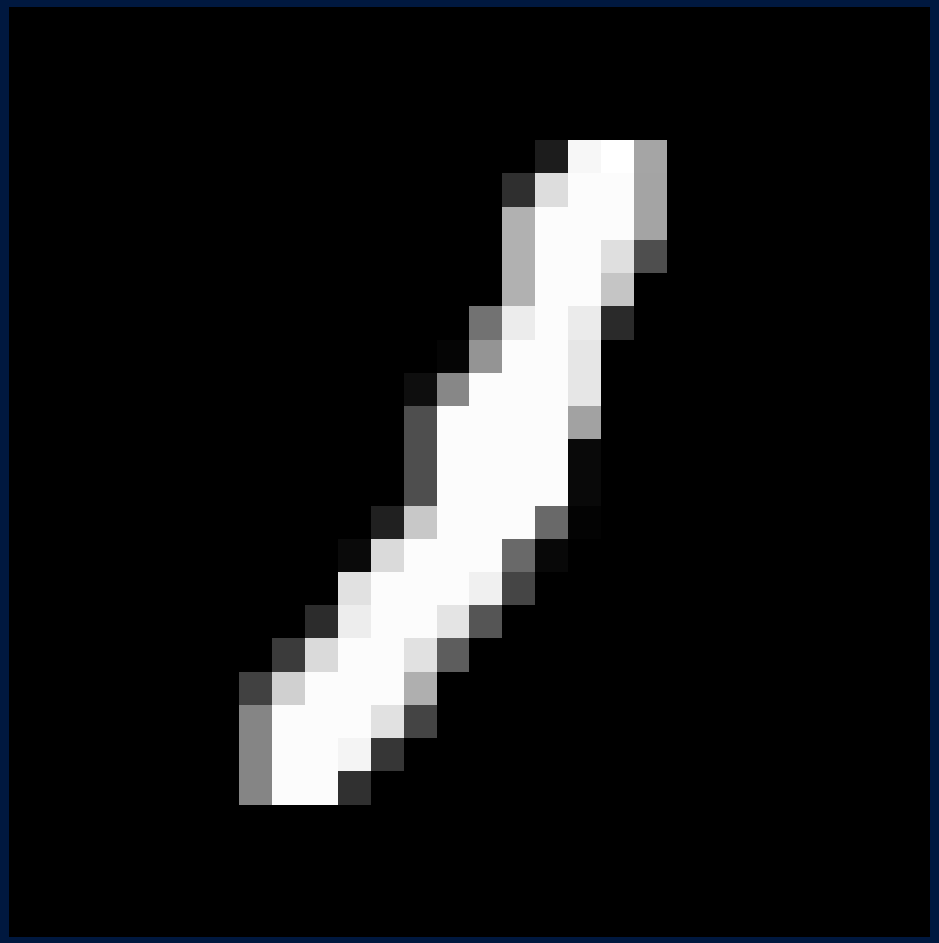
Human Neural Network Activity



Human Neural Network Activity



Human Neural Network Activity



10 Minute break

Returning at x:xx

Next up: Convolutional Neural Networks

Activity Discussion

This activity demonstrates how a fully connected network operates

There was one part of this exorcise will look ahead to our next topic. Can anyone identify it?

Can you think of any ways to improve our HNN’s performance?

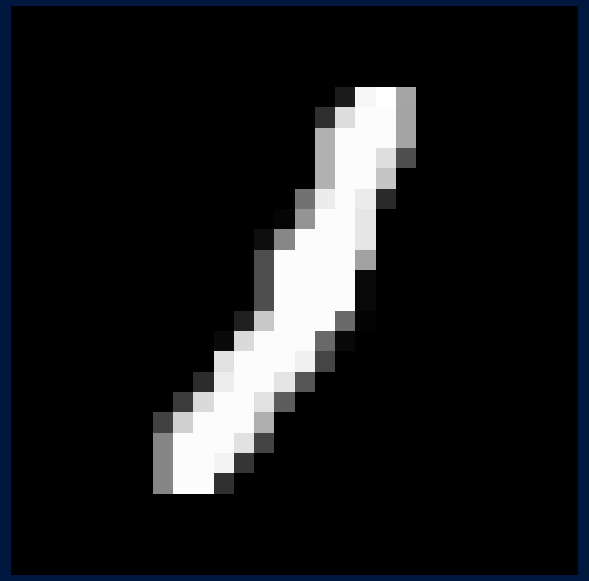
Putting it all together

How do fully connected networks work in practice?

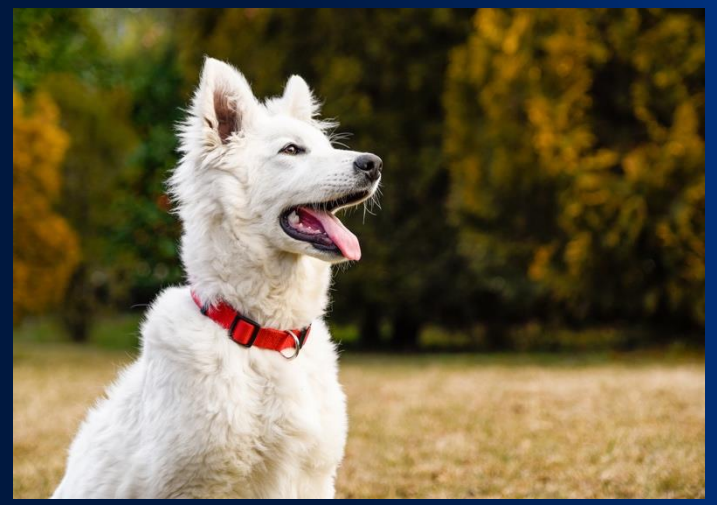
https://okai.brown.edu/chapter6.html

What are some major limitations of Fully Connected Layers?

1. Scalability
2. Scalability!
3. Have I mentioned scalability?
4. Restricted Input



VS



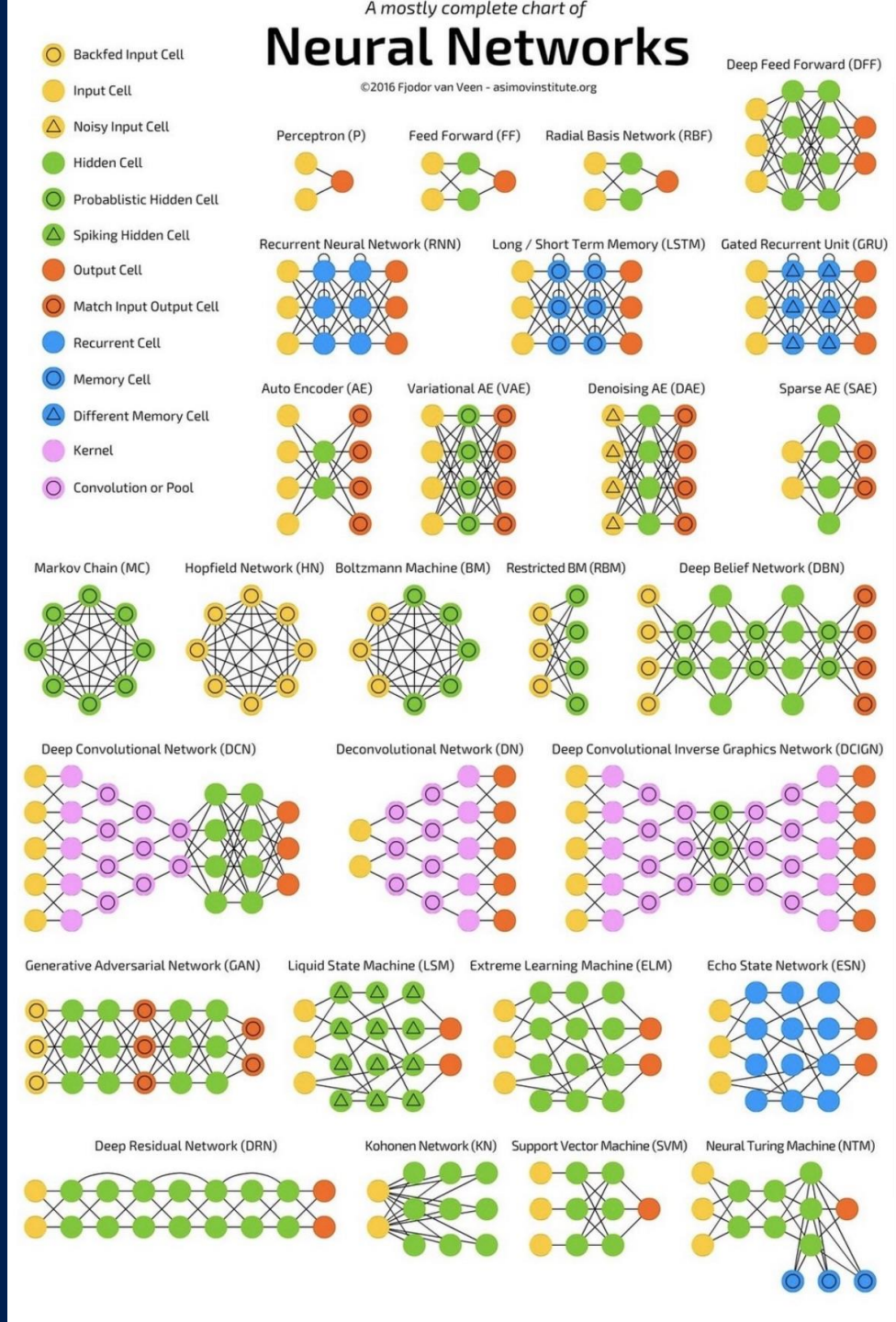
Many networks end in one or more fully connected layer(s).

What we discuss the rest of today and tomorrow will by and large be ways to take input that is too large/amorphous to go directly into a Fully connected network and finding a “feature representation” that accurately represents that input

Review

Fully connected Networks

- Built from multiple regression models combined together
- Can solve complex non-linear problems
- Still used as a fundamental part of many more advanced types of network
- Need to represent input data as a vector of fixed size



The Jackson Laboratory

Neural Networks: Convolutional Neural Networks

Finding patterns in matrixes

Learning Goals:

What are Convolutional Neural Networks (CNNs) and how do they build on fully connected neural networks ?

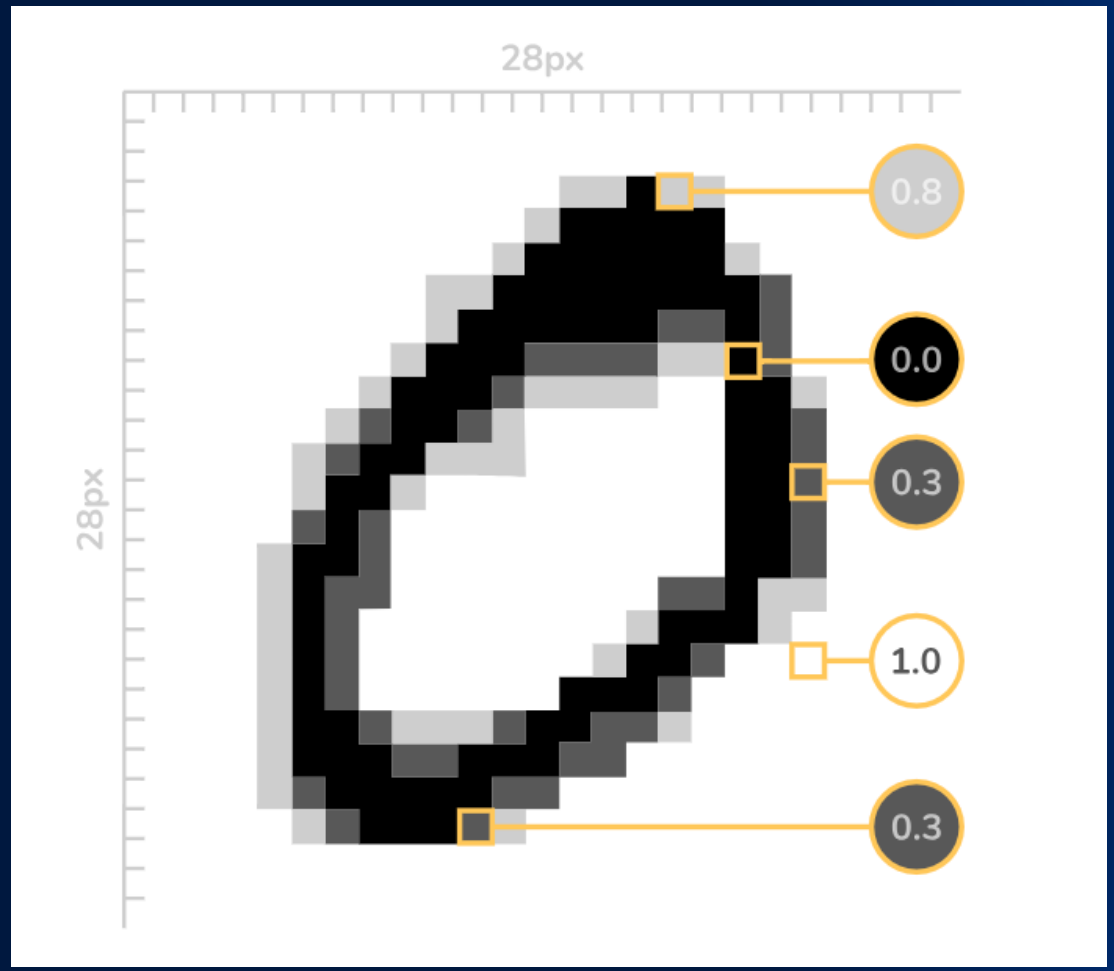
You will be able to:

- Explain what a convolution does
- Contrast a convolutional layer to a fully connected layer
- Determine when to use convolutional layers

A brief aside: An image is worth HxW numbers

Think of your computer monitor

- Anything you see on it is thousands of little points of light (pixels)
- Color is just a combination of red green and blue points of light
- We know how bright each point is
- We can represent any image as a rectangle of numbers aka a matrix



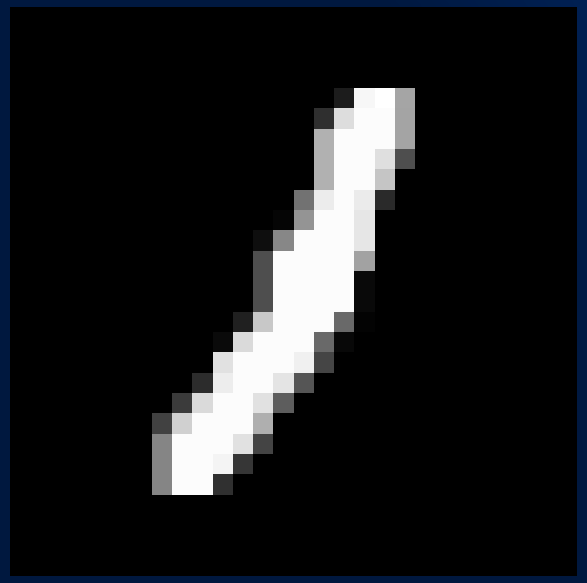
Why do we need convolutions?

MNist has 28pxl X 28 pxl gray scale images

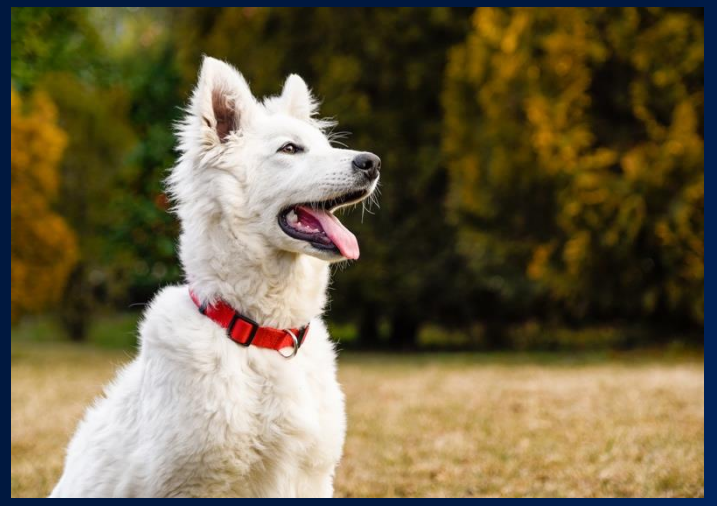
- Each image can be flattened into a 784-element vector

This picture of a dog is 512pxls X 910pxls and in color.

- If we flattened it, it would be nearly five hundred thousand numbers
- We need a way to accurately describe the image with far fewer numbers



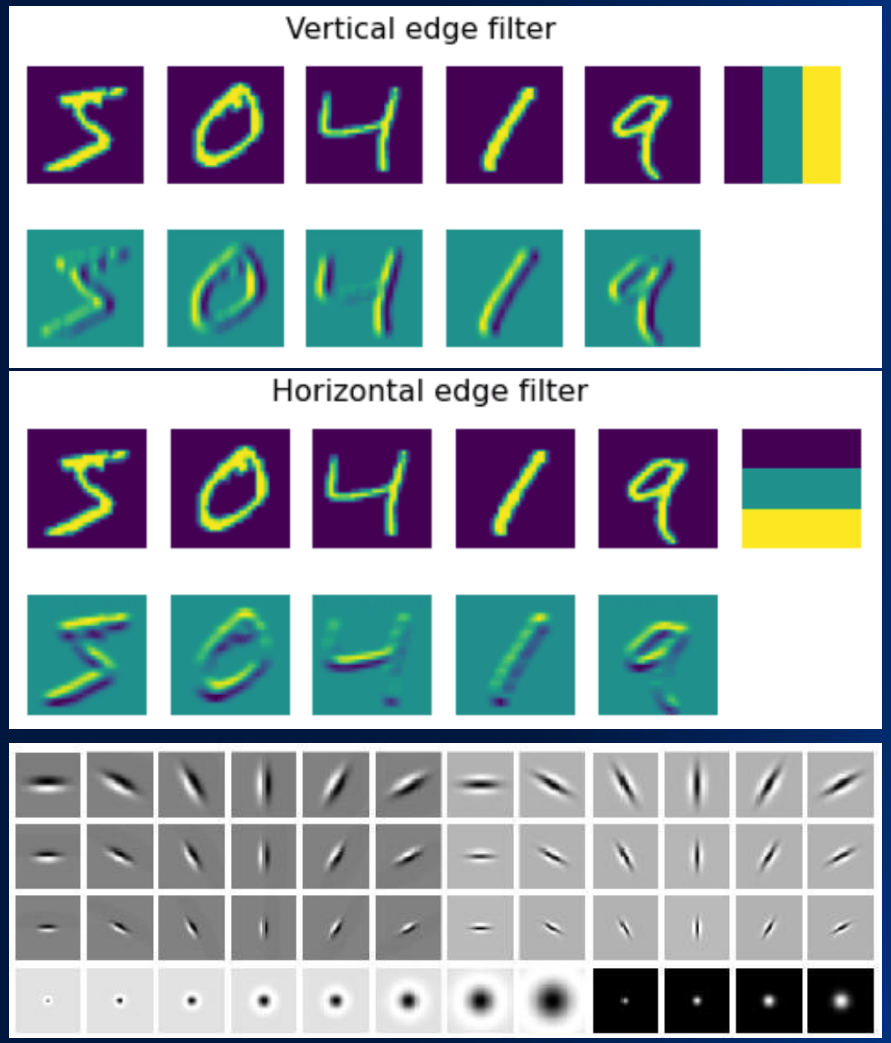
VS



What is a convolution?

Convolutions are filters that look at small patches of an image to find patterns

- We can create filters manually to find specific shapes
  - This is what the input layer was doing in the HNN activity
- Instead of telling the network what to look for, we can let it learn what shapes are important just like we did with the perceptrons



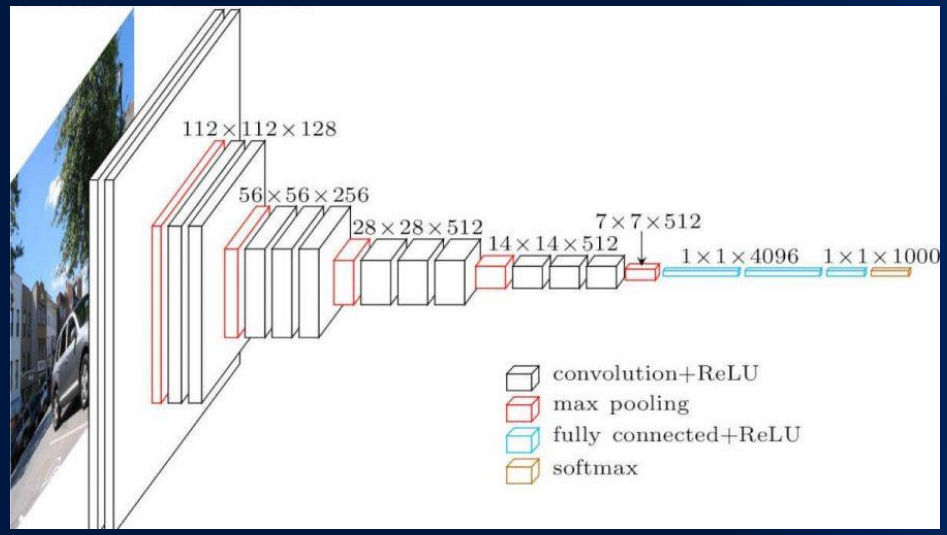
How do we find more complex shapes?

Multiple perceptrons ➔ Fully connected layer

Multiple FCs ➔ Fully connected network

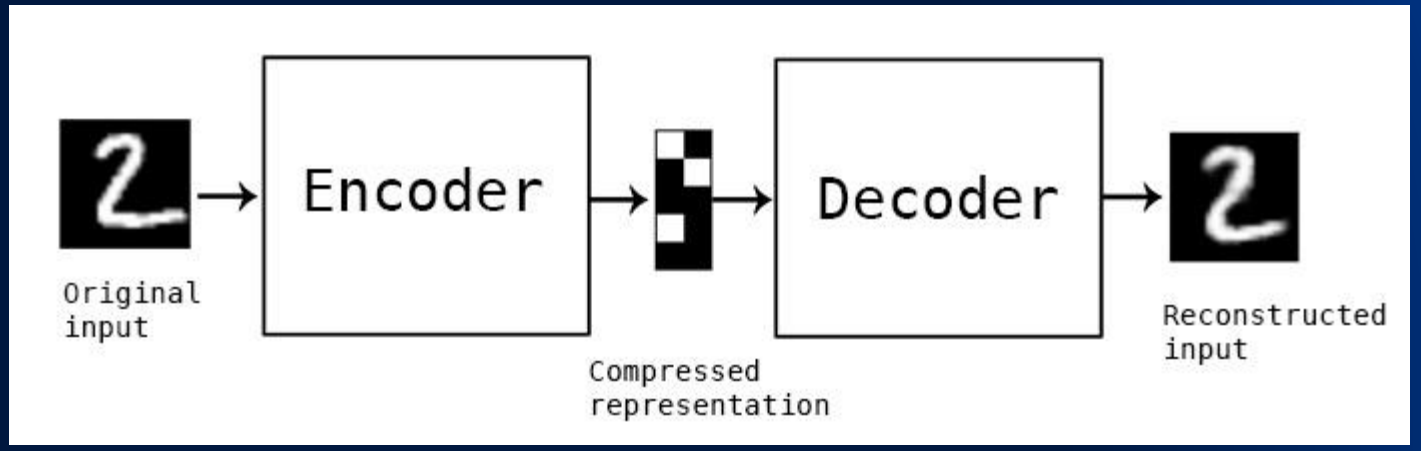
Multiple convolutions ➔ Conv layer

Multiple conv layers ➔ Convolutional NN



What about going in reverse?

Deconvolutions are the inverse of convolutions



- Convolutions take a series of patterns and turn an image into a description of that image
- Deconvolutions take a description and “draw” an image based on it.
- An alternative to ending a network in a fully connected layer when you want a matrix instead of a vector. E.g. image generation

To discuss after the activity

This game demonstrates using simplified patterns to describe complex images

What kinds of shapes were more informative?

Were shapes from certain parts of the image more informative than others?

Contour Classification Activity

1. Same roles as the Human Neural Network activity
2. See Hand out for further instructions

Activity Discussion

This game demonstrates using simplified patterns to describe complex images

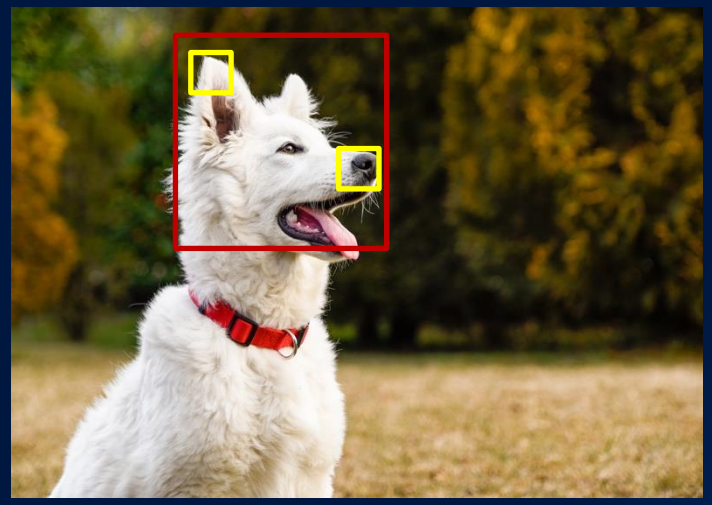
What kinds of shapes were more informative?

Were shapes from certain parts of the image more informative than others?

What are some major limitations of convolutional layers?

1. No memory
    - Imagine working with a video instead of an image
    - There is temporal context the convolution can not see

2. Limited field of view
    - Because of the limited size of a convolution, you have limited context for fine details



3. Fixed number of channels

Review

Images can be thought of as matrixes of numbers

Convolutional neural networks

Strengths:

- Can efficiently summarize large input matrixes
- Can be reversed to (re)create an image

Limitations:

- No memory
- Limited field of view
- Fixed number of channels

10 Minute break

Returning at x:xx

Next up: Recurrent Neural Networks &
Natural language Processing

Neural Networks: Recurrent Neural Networks

Working with Sequences

Learning Goals:

What are Recurrent Neural Networks (RNNs) and how do they allow us to work with sequential data such as text?

You will be able to:

- Tokenize input data
- Explain why you would add a loop to a network
- Infer the limitations/downsides of Recurrent layers

A brief aside: Tokenization

Q: How do we perform calculations on data that isn't numbers?

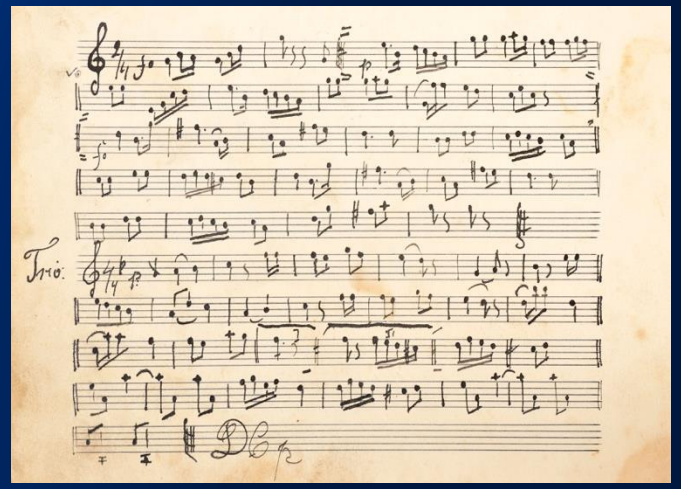
A: A bit of a trick question, because we can't do so directly.

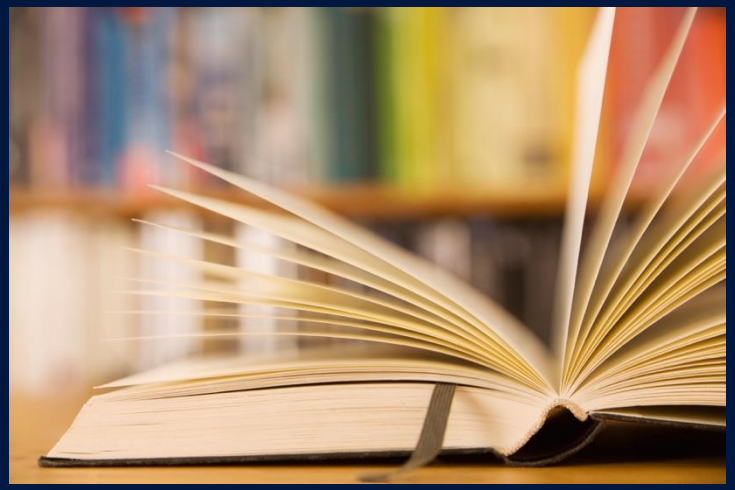
But we can use numbers to represent non-numeric data.

Types of tokenization used in AI/ML:

- Word level
- Character level
- Sub-word level

What sorts of data are sequential?





How is Sequential data different?

Order Matters

- Earlier parts of sequential data provide context for later parts
- Imagine hearing the punchline of a joke before the setup (or without the set up at all)
- Consider how the meaning of this sentence changes if we change the order of two words

Eve spied on Alice

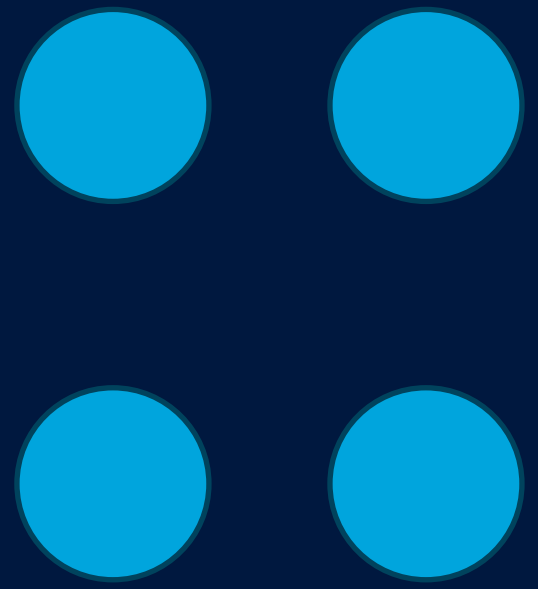
What problems does this create?

We often don’t know the size of our input

- Two sentences can have different numbers of words.
- This means each token must be considered individually

“feed forward” networks only see a snapshot of what is going on

- They possess no memory of previous inputs

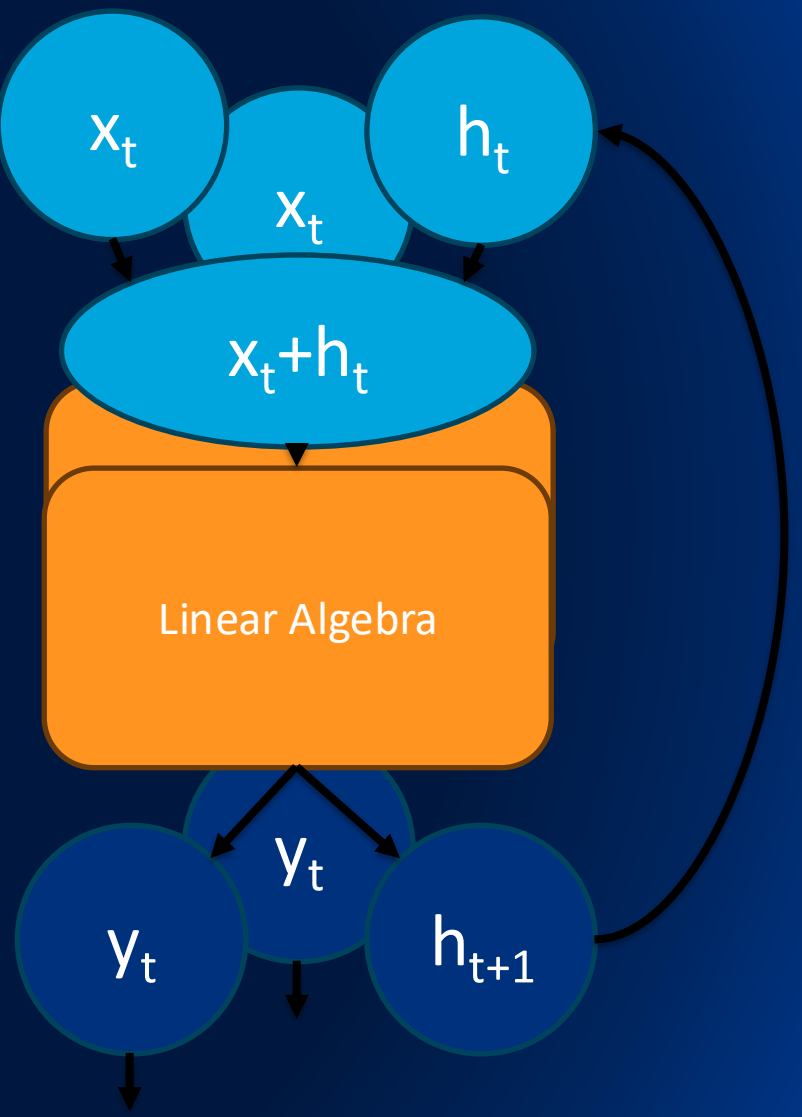


How can we give our network a memory?

Recurrent layers to the rescue

We create a new vector **h** that can store information for the layer

Each input **x** is connected to **h** and fed through the layer to create an output **y** and a new **h** that is used with the next input



To Discuss after the activity

This game demonstrates how considering the sentence as a sequence improves interpretability

Can you think of any limitations with considering the sentence in this way?

Can you think of a method that would improve on this sequential processing?

Sentence Analysis Activity

1. Break into small groups
2. Each group gets a stack of index cards
    - The top card is a random word from the target sentence
    - The rest of the cards are the sentence written out sequentially
3. Try and guess the meaning of the sentence from the top card
4. Flip each of the remaining cards over one by one and try to guess the meaning of the sentence in as few cards as you can

Activity Discussion

This game demonstrates how considering the sentence as a sequence improves interpretability

Can you think of any limitations with considering the sentence in this way?

Can you think of a method that would improve on this sequential processing?

What are some major limitations of recurrent layers?

1. Limited memory
   - There is an upper limit to how much information can be stored in **h**
   - This puts an upper limit on number of tokens the network can consider
2. Recency bias
   - Because **h** changes with each input the networks memory prioritizes more recent tokens
3. No token prioritization
   - The network has no way of learning which tokens are more important
4. Limited parallelizability
   - Whenever you treat something as a sequence you have to process it in order limiting your ability to split the work across multiple processors

What are some major limitations of recurrent layers?

1. Limited memory
    - There is an upper limit to how much information can be stored in **h**
    - This puts an upper limit on number of tokens the network can consider

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

V
S

What are some major limitations of recurrent layers?

2. Recency bias
    - Because **h** changes with each input the networks memory prioritizes more recent tokens

The quick brown fox jumps over the lazy dog

What are some major limitations of recurrent layers?

3. No token prioritization
    - The network has no way of learning which tokens are more important

The quick brown fox jumps over the lazy dog

What are some major limitations of recurrent layers?

4. Limited parallelizability
    - Whenever you treat something as a sequence you have to process it in order limiting your ability to split the work across multiple processors

Review

Tokenization

- By representing things like words with vectors of numbers we can input them into neural networks

Recurrent neural networks

Strengths:

- Adds memory to a neural network
- Incorporates sequential context
- A lot of things are sequential

Limitations:

- Computationally expensive
- Limited memory capacity and range
- Can’t learn relative importance of tokens

Learning Goals:

How do we Generate new things with a neural network and what does a state-of-the-art network look like?

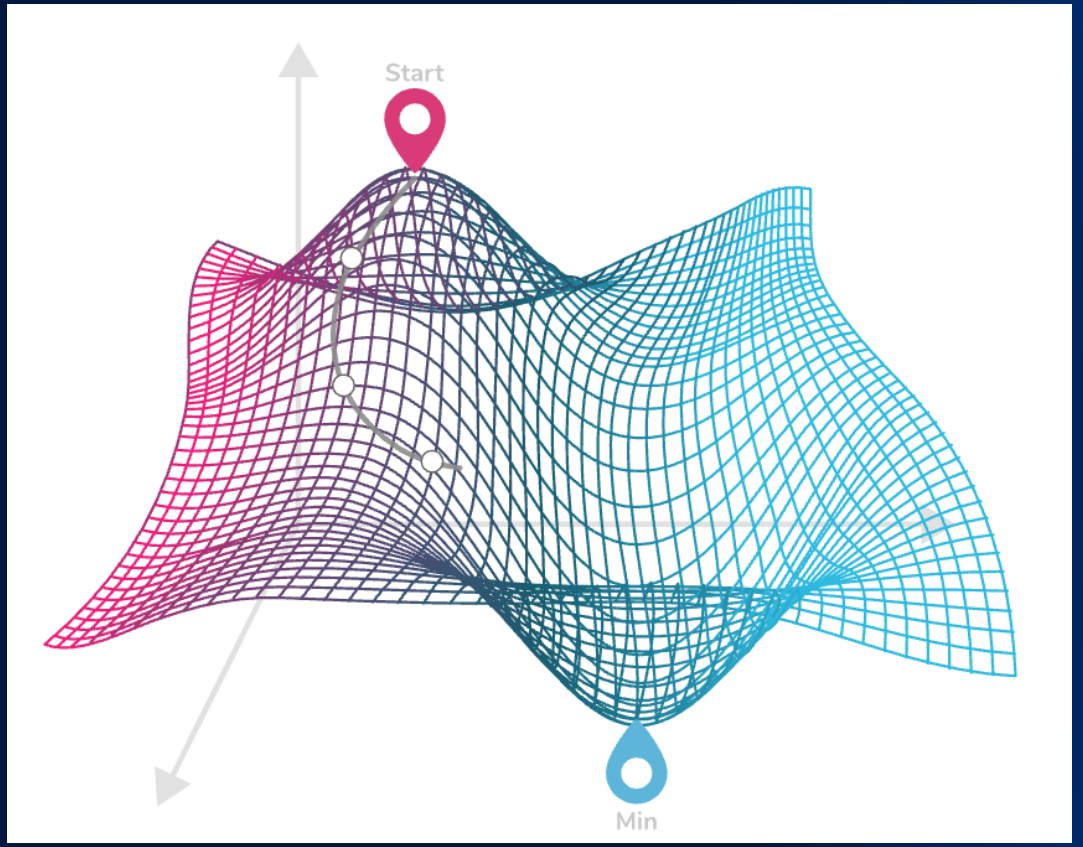
You will be able to:

- Contrast generative vs. discriminative / predictive models; outline GAN generator–discriminator interplay.
- Explain at a high level how self-attention works and why it replaced recurrence for many tasks.
- Appreciate the power (and pitfalls) of large-scale pre-training through guest talk & games.

Call Back: Loss Functions

A good loss function is necessary to train a neural network

- Think back to the treasure splitting game from day 1
- A loss function measures how different your output is from the desired output
- This is easy when you have a single right answer for each input



How do we quantify the quality of generated content?

Q: When generating something “new”, how can we compute the loss?

- What are we trying to quantify?
  - Plausibility
- How can we measure that?
  - What if we could measure how good our generator is at tricking someone or something?

To consider: what implications does defining loss in this way have for AI Hallucinations

Can we turn generation into a prediction problem?

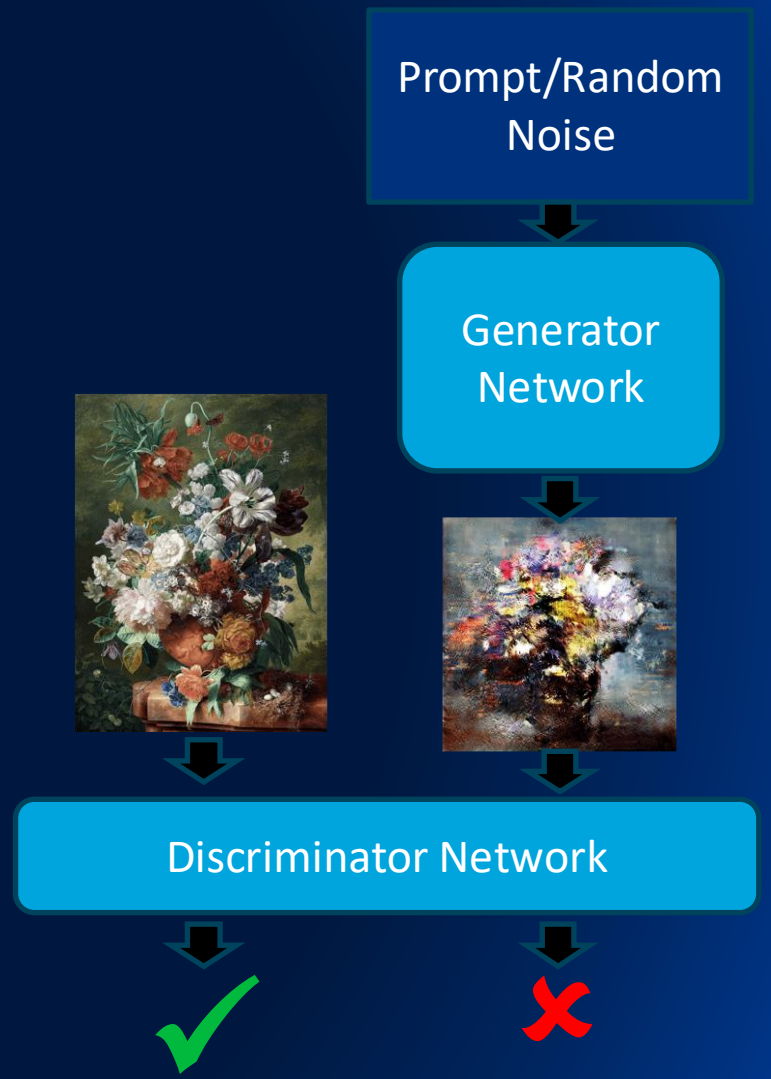
Generative Adversarial Networks have two parts:

- The Generator which creates something new

  Examples:

  - Deconvolutions to create an image
  - A Recurrent Network that chooses a plausible next word for a sentence

- The Discriminator which takes the generated output and a real example and tries to determine which is the fake



To discuss after the activity

This activity puts you in the role of generators and discriminators

Generators: what strategies did you use to improve the plausibility of the answers as the game went on?

Discriminators: were you basing your decision on anything other than plausibility?

Everyone: what implications would training a network in this way have, if there were no correct answer?

Data science balderdash

1. Split into pairs and decide on who will be the generator and who will be the discriminator. (in the case of an odd number of players one group should have 2 discriminators)
- Generators:
    2. take one of the stacks of index cards, on one side is a data science themed question on the other is the real answer.
    3. Make up a fake answer to the question
    4. Tell the discriminator(s) both your fake answer and the real answer
- Discriminators:
    5. Guess which answer is the real one
6. Keep playing more rounds until time is called or you run out of cards (optionally switching roles)

Activity Discussion

This activity puts you in the role of generators and discriminators

Generators: what strategies did you use to improve the plausibility of the answers as the game went on?

Discriminators: were you basing your decision on anything other than plausibility?

Everyone: what implications would training a network in this way have, if there were no correct answer?

What are some major limitations of GANs?

1. Limited by the capabilities of the generator and discriminator architectures
2. Hard to train and can't be "transferred" easily

Only a limitation in some cases:

3. Correctness is only rewarded in so far as it makes the generated answer more plausible
4. Will always generate an output

VS

Review

Generative Adversarial Networks

- Allows quantifying loss for generated output
- Turns a generation problem into a classification problem
- Trained to produce plausible output
- Will always produce some form of output